

SRS

# 소프트웨어 요구사항 명세서

통합 게임 데이터 및 하드웨어 최적화 플랫폼

v1.0 · 2026.03 · 2021663046 이건우

## 변경 이력

| 버전     | 작성/수정일     | 주요 수정 내용                    | 작성자 |
|--------|------------|-----------------------------|-----|
| v1.0.0 | 2026.03.27 | 초안 작성 및 매칭 알고리즘/인터페이스 상세 정의 | 이건우 |

# 1. 소개 (Introduction)

## 1.1. 제품 범위 및 가치 제안

본 플랫폼은 스팀(Steam) 및 라이엇 게임즈(Riot Games) 등 다수의 유통 플랫폼으로 파편화된 사용자의 게임 데이터(누적 플레이 시간, 업적 등)를 단일 대시보드에 통합하고, 사용자의 하드웨어 제원에 최적화된 그래픽 설정값을 제공하는 것을 목적으로 합니다.

핵심 가치: PC 플랫폼의 하드웨어 파편화로 인한 최적화 설정 탐색 비용을 최소화하며, 시스템 병목(Bottleneck) 현상을 정밀 진단하여 최적의 게임 환경을 구축합니다.

## 1.2. 대상 독자

| 역할            | 본 문서 활용 목적                                                               |
|---------------|--------------------------------------------------------------------------|
| 개발자 / 시스템 설계자 | 시스템 아키텍처, 데이터 처리 로직 및 RESTful API 규격을 참조하여 DB 스키마 설계 및 엔드포인트 구현의 기준으로 활용 |
| QA / 테스터      | 수락 기준(Acceptance Criteria)을 바탕으로 기능 정상 동작 여부와 비기능적 품질 속성 검증              |
| 비즈니스 분석가 (BA) | 비즈니스 모델이 기술적 요구사항으로 정확히 변환되었는지 확인하고 프로젝트 범위 관리                           |

## 1.3. 시스템 정의

| 데이터 중개 플랫폼                                         | 커뮤니티 최적화 허브                                                     |
|----------------------------------------------------|-----------------------------------------------------------------|
| 외부 게임사의 원시 데이터를 수집·가공하여 단일 진실 공급원(SSoT) 역할을 수행합니다. | 하드웨어 벤치마크 데이터를 기반으로 최적의 설정값을 Power User와 General User 간에 매칭합니다. |

## 2. 시스템 및 기능 요구사항 (System & Functional Requirements)

### 2.1. 요구사항 기술 규칙

| 항목 | 내용                                                                   |
|----|----------------------------------------------------------------------|
| 표준 | 모든 기능 요구사항은 EARS(Easy Approach to Requirement Syntax) 구문을 엄격히 준수합니다. |
| 형식 | WHEN [이벤트 발생], the [시스템] shall [동작]                                  |

### 2.2. 핵심 기능 상세 명세

#### REQ-001 외부 계정 연동 및 데이터 이관

WHEN 사용자가 OAuth 2.0 인증을 통해 외부 플랫폼(Steam, Riot Games) 계정을 연동할 때

SHALL 외부 Web API를 호출하여 사용자의 게임 목록 및 업적 데이터를 수집하고 이를 RDBMS에 저장해야 한다.

| 입력                                                         | 처리 로직                                                                                                                                   | 출력                                                                        |
|------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| OAuth Access Token User Platform ID API Key (Steam / Riot) | 외부 API(Steam/Riot) 엔드포인트 비동기 호출 수신된 JSON에서 App_ID, Playtime_Forever, Achievement_Status 필드 추출 기존 DB 내 사용자 식별자와 매핑하여 Update 또는 Insert 수행 | HTTP 200 OK Sync_Status: Success Last_Sync_Timestamp 연동된 게임 리스트 객체 (JSON) |

#### REQ-002 하드웨어 프로필 정규화 및 등록

WHEN 사용자가 하드웨어 자원(CPU, GPU, 해상도)을 직접 입력하거나 수정할 때

SHALL 입력된 텍스트를 표준 하드웨어 DB 명칭과 정규화(Normalization)하여 사용자 프로필에 저장해야 한다.

| 입력                                                              | 처리 로직                                                                                                   | 출력                                                 |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| CPU 모델명 (String) GPU 모델명 (String) 해상도 (Int x Int) 주사율 (Int, Hz) | 사용자 입력 문자열에 대한 정규표현식(Regex) 검사 하드웨어 마스터 테이블과의 유사도 검색을 통한 표준 명칭 매칭 User_Hardware_Profile 테이블에 정규화된 키값 저장 | Profile_Update_Result: Success 정규화된 하드웨어 모델명 ID 반환 |

#### REQ-003 가중치 기반 알고리즘 최적화 매칭

WHEN 사용자가 특정 게임의 최적화 설정을 요청할 때

SHALL DB 내 프로파일 중 사용자의 하드웨어 환경과 90% 이상 일치하는 환경에서 작성된 설정값을 우선순위로 정렬하여 전송해야 한다.

| 입력                                       | 처리 로직                                                                                                                    | 출력                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Target_Game_ID User_Hardware_Spec_Object | 가중치 기반 유사도 점수 계산: CPU 세대 및 코어 수: 20% GPU 등급 및 VRAM: 50% 해상도 및 주사율: 30% 총합 점수 0.9 이상 레코드가 필터링 Average_FPS 기준 내림차순 정렬 후 반환 | Ranked_Settings_List (JSON) 각 설정별 상세 그래픽 옵션값 및 예상 FPS |

#### REQ-004 데이터 시각화 엔진

WHEN 통합 대시보드 페이지가 로딩될 때

SHALL 수집된 누적 플레이 시간 데이터를 활용하여 기간별 시계열 차트 및 업적 달성률 현황을 렌더링해야 한다.

| 입력                                  | 처리 로직                                                                     | 출력                                                        |
|-------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------|
| User_Play_Logs<br>Achievement_Stats | Recharts 라이브러리 인터페이스에 맞춘 데이터 배열 가공<br>일간 / 주간 / 월간 단위 데이터 집계(Aggregation) | 동적 시각화 대시보드 Line<br>Chart (시계열 추이) Pie<br>Chart (플랫폼별 비율) |

REQ-003 가중치 상세

| 가중치 항목        | 비율  | 비고                    |
|---------------|-----|-----------------------|
| GPU 등급 및 VRAM | 50% | 병목의 주요 원인 — 가장 높은 가중치 |
| 해상도 및 주사율     | 30% | 동일 해상도 환경 우선 매칭       |
| CPU 세대 및 코어 수 | 20% | 보조 필터링 기준             |

2.3. 관리 기능

| 기능        | EARS 요구사항                                                             |
|-----------|-----------------------------------------------------------------------|
| 시스템 모니터링  | WHEN 관리자가 콘솔에 접속할 때 → 트래픽 통계, 하드웨어 프로파일 등록 현황 및 API 에러 로그 조회 인터페이스 출력 |
| 사용자 권한 통제 | WHEN 관리자가 특정 사용자 상태를 변경할 때 → Access_Level 필드 업데이트 및 접근 권한 즉각 제한       |

### 3. 외부 인터페이스 요구사항 (External Interface Requirements)

---

#### 3.1. 사용자 인터페이스 (UI)

| 항목       | 명세                                                                 |
|----------|--------------------------------------------------------------------|
| UX 연속성   | React.js 기반 SPA(Single Page Application) 구조 채택 — 페이지 전환 시 상태 유실 방지 |
| 반응형 레이아웃 | Tailwind CSS Utility-first 그리드 시스템 — 모바일 / 태블릿 / 데스크톱 전 뷰포트 대응     |

#### 3.2. 소프트웨어 인터페이스

| 레이어               | 기술 스택 / API                                         |
|-------------------|-----------------------------------------------------|
| Backend Interface | Node.js (Express.js 프레임워크) — JSON 포맷 RESTful API 통신 |
| Steam Web API     | IPlayerService/GetOwnedGames 등 인터페이스를 통한 게임 데이터 수신  |
| Riot Games API    | 전적 및 사용자 활동 데이터 수집 엔드포인트 연동                         |

#### 3.3. 하드웨어 인터페이스

웹 환경의 보안 제약으로 인해 하드웨어를 직접 탐색하는 대신, 사용자가 입력한 모델명을 표준 DB와 매칭하는 검색 · 선택 인터페이스를 제공하여 데이터의 무결성을 보장합니다.

## 4. 비기능 요구사항 (Non-Functional Requirements)

### 4.1. 성능 및 용량 (Capacity & Scalability)

| 항목     | 요구 사항                                                |
|--------|------------------------------------------------------|
| 비동기 처리 | 외부 API 호출 시 비동기(Asynchronous) 패턴 적용 — 서버 블로킹 방지      |
| 캐싱 전략  | Redis 등 메모리 캐시 활용 → 빈번한 조회 요청 100ms 이내 응답 유지         |
| 확장성    | Stateless 아키텍처 기반 Auto-scaling 지원 — 수평적 Scale-out 설계 |

### 4.2. 보안성 (Security)

| 항목      | 요구 사항                                       |
|---------|---------------------------------------------|
| 인증 표준   | 모든 외부 계정 연동 및 로그인 — OAuth 2.0 프로토콜 준수       |
| 데이터 암호화 | 사용자 개인정보(PII) 및 API 키 — AES-256 알고리즘 암호화 저장 |

### 4.3. 신뢰성 및 가용성 (Reliability & Availability)

| 항목      | 요구 사항                                                            |
|---------|------------------------------------------------------------------|
| 서비스 가동률 | 연간 99.9% 이상 가동률 목표. 장애 시 캐시된 데이터 우선 노출 → Graceful Degradation 전략 |
| 우선순위 할당 | 프리미엄 구독 사용자의 데이터 동기화 요청 → API 호출 큐(Queue) 우선순위 할당                |

### 4.4. 호환성 및 유지보수성 (Compatibility & Maintainability)

| 항목       | 요구 사항                                                     |
|----------|-----------------------------------------------------------|
| 브라우저 호환성 | HTML5 표준 준수 — Chrome, Safari, Edge에서 일관된 동작 보장            |
| 유지보수성    | Winston + Morgan 라이브러리 기반 상세 로깅 체계 구축. 모듈형 아키텍처로 기능 확장 용이 |

## 5. 기술 스택 제약 및 환경적 가정 (Constraints & Assumptions)

### 5.1. 확정 기술 스택

| Frontend                       | Backend            | Database                               |
|--------------------------------|--------------------|----------------------------------------|
| React.js Tailwind CSS Recharts | Node.js Express.js | PostgreSQL (복잡한 관계 모델 ·<br>가중치 쿼리 최적화) |

### 5.2. 환경적 가정

Steam 및 Riot Games API가 중단 없이 제공되며, 개발자 정책에 따른 데이터 접근 권한이 유효함을 전제로 합니다.

## 6. 수익 창출 및 향후 계획 (Monetization & Roadmap)

### 6.1. 비즈니스 모델

| 모델         | 구현 방안                                                       | 단계      |
|------------|-------------------------------------------------------------|---------|
| 하드웨어 타겟 광고 | User_Hardware_Profile 분석 → 성능 병목 예상 부품에 업그레이드 제안 및 구매 링크 노출 | Phase 1 |
| 프리미엄 구독    | 인게임 설정 파일(Config) 클라우드 백업/복원 + Power User와의 심화 성능 비교 데이터 제공 | Phase 1 |
| 익명화 데이터 판매 | 대규모 유저의 실제 FPS 지표 통계화 → B2B 리서치 리포트 형태 판매                   | Phase 2 |

### 6.2. 확장 로드맵

Phase 2 목표: 에픽게임즈(Epic Games) 및 콘솔 플랫폼(PSN, Xbox Live) API 연동을 위한 모듈형 커넥터 아키텍처 추가 개발



## 7. 부록 (Appendix)

### 7.1. 용어 사전 (Glossary)

| 용어                            | 정의                                        |
|-------------------------------|-------------------------------------------|
| 인증된 사용자 (Verified User)       | OAuth 인증을 통해 하나 이상의 외부 게임 계정 연동을 완료한 사용자. |
| Power User                    | 벤치마크 데이터 및 최적화 설정값을 직접 등록하는 고급 사용자.       |
| General User                  | 타인의 설정값을 조회·적용하는 일반 사용자.                  |
| SSoT (Single Source of Truth) | 모든 데이터의 단일 원본 저장소. 데이터 일관성 보장.            |
| Bottleneck                    | CPU/GPU 간 성능 불균형으로 전체 시스템 성능이 제한되는 현상.    |